

The I2S-pro family: two reasons input-to-state copying gets through

Backdrop. *I2S-pro* collects the roadblocks where a fuzzer that *watches the program's comparisons and pastes the observed value back into the input* (cmplog's input-to-state, I2S) gets through, while a byte-mutation fuzzer stays stuck. Picture a lock whose key is lying right next to it: I2S notices the key and uses it; plain fuzzing has to guess. Both categories below share that technique — they differ only in whether the key is a *string* or a *number*. Counts are validated roadblocks (n). (Two buckets we initially split off — *last-mile operand value precision* and *structural assembly depth* — turned out to be the *same* exact-literal substitution and are folded into category 1 as sub-cases below.)

1. Exact string / literal gate i2s_string_literal_substitution ($n = 199$)

The program demands an exact run of bytes: a magic header, a format tag (IDAT, sRGB), a protocol token (HTTP/, ://). I2S sees that literal in the comparison and drops it straight into the input; random mutation would essentially never spell it out. *The key was lying beside the lock.* This is the dominant case.

```
// lcms cmspcs.c : _cmsLCMScolorSpace()
// dispatch on the 4-byte ICC color-space signature 'sig'
switch (sig) {
case cmsSigGrayData: return PT_GRAY;    // sig == 'GRAY'
case cmsSigCmykData: return PT_CMYK;    // <-- blocker (lcms/2246): sig ==
    'CMYK'
}
```

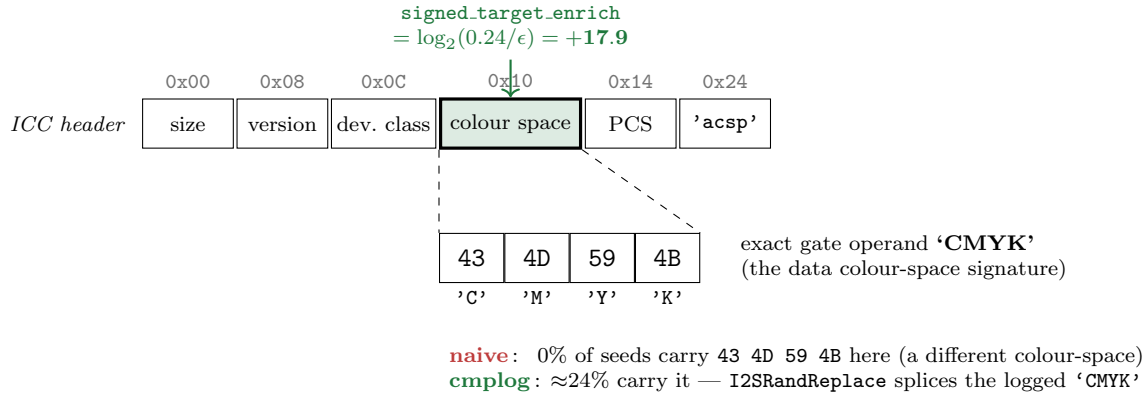
How we know it is really I2S. If I2S is planting the literal, the cmplog corpus should be *enriched* in those exact bytes at the gate offset, relative to a naive corpus. We measure that enrichment as a $\log_2$ frequency ratio; the rule asks for at least a $2\times$ enrichment.

Tool: operand_enrichment

Metric: $\text{signed_target_enrich} = \log_2 \frac{\text{cmplog freq} + \epsilon}{\text{naive freq} + \epsilon}$, $\epsilon=10^{-6}$ (> 0 = enriched, < 0 = depleted)

Verify rule (the arbiter fires this, no LLM): $\text{signed_target_enrich} \geq 1.0$

Real example: lcms/2246 (_cmsLCMScolorSpace) — at the operand offset the 4-byte signature is *absent* from the naive corpus (0%) but reaches $\approx 24\%$ in the cmplog corpus, so $\text{signed_target_enrich} = \log_2(0.24/\epsilon) = +17.9$; the decoy bytes are *depleted* ($\text{decoy_enrich} = -1.1$).



The gate operand of `1cms/2246` is the ICC *data colour-space signature* at offset `0x10`; `operand_enrichment` measures how often each fuzzer's corpus carries the exact bytes *there*. Header field offsets are the ICC format; the measured signal is `signed_target_enrich` = +17.9 (naive places the literal in 0% of its seeds), of which $\approx 24\%$ is the implied cmplog operand-offset rate.

Sub-case: the literal *floats* (no fixed offset). Three of these branches place the literal at a *content-dependent* position rather than a fixed file offset — an XML entity name, a comment marker — so the enrichment test above cannot be anchored. The mechanism is identical (paste the exact literal); only the operand's position differs, so we confirm them by the Layer-1 cmplog $\succ$ naive advantage plus the *absence* of any fixed-offset signature (`gate_signature == false`). Real example (`libxml2, xmlGetPredefinedEntity, entities.c:307`):

```

switch (name[0]) {                                     // name = a parsed entity
    reference
case 'a':                                             // <-- blocker: needs the "amp
    "/"apos" name
    if (xmlStrEqual(name, "amp")) return &xmlEntityAmp;
    if (xmlStrEqual(name, "apos")) return &xmlEntityApos;
    break;
default: return NULL;                                // naive lands here on all 479
    of its calls
}

```

cmplog takes `case 'a'` 63 times by splicing the logged `"amp"/"apos"` operands into the input; naive never spells them. The literal sits wherever the `&...`; reference occurs, so there is no fixed offset to measure — hence the no-signature confirmation.

Sub-case: confirmed by the value-distance route (6 branches). Six branches (5 `curl`, 1 `harfbuzz`; operands `"Host:"`, `"://"`, `"pop3"`, `"GPOS"`) are the *same* exact-literal mechanism but could not be anchored for the byte-frequency enrichment test, so the arbiter confirmed them with `value_distance_reached` instead: the cmplog (I2S) corpus lands the exact operand (residual 0) while the *blind* non-I2S corpus stalls at a positive Hamming residual. This is exactly category 1 — “paste the exact bytes” — measured by closure-of-distance rather than enrichment (the same situation as the floating-offset sub-case above, where the enrichment test also could not be anchored). Real example (`curl/129, strncmp(line, "Host:", 5)`): the I2S side closes the whole `distance_gap` = 0.8 to the operand (`distance_closure_ratio` = 1.0), the blind side stalls 0.8 away.

Tool: <code>value_distance_reached</code>
Metric: <code>distance_gap</code> = how far the non-I2S corpus’s nearest seed stalls from the target operand; <code>distance_closure_ratio</code> = fraction of that gap the I2S corpus closes (1.0 = lands it exactly)
Verify rule (the arbiter fires this, no LLM): <code>winner_closer == true AND distance_gap > 0 AND distance_closure_ratio >= 0.5</code>
Real example: <code>curl/129</code> , operand <code>"Host:"</code> — <code>distance_gap= 0.8, distance_closure_ratio= 1.0</code> : the I2S side lands the literal, the blind side never does. (<i>Earlier split off as “last-mile value precision”; folded into category 1, 2026-06-25 — the rule compares I2S vs. naive, i.e. plain literal substitution, not a distinct mechanism.</i>)

Sub-case: the operand sits behind a multi-stage container (8 branches). Eight branches (`bloaty/87,90`, `harfbuzz/6236`, `lcms/2113`, `libpng/3820,3965`, `libxml2/6909,7287`) reach the gate only after several *interlocking* fields validate — an ELF section table, a PNG chunk grammar, an XML scheme+attribute. We initially split these off as “structural assembly depth,” but the move is still the *same* one: I2S pastes the exact field literal(s); there are simply *several* operands scattered across a multi-stage container rather than one. “Multi-operand” is a *description*, not a separate mechanism — and crucially *no cheap signal distinguishes it from single-operand category 1*. The original `depth_reach` metric (`depth_lift` = winner/loser per-seed executions of the blocker line) is *loop-confounded* — `bloaty/90`’s “blocker” is literally `for (...; i < elf.section_count(); i++)`, so the $2375\times$ “depth” is just loop trips, not assembly — and *near-tautological* (it runs the already-arriving resolving/blocking seeds). Byte-offset enrichment cannot anchor (the fields float, 5/8 have `gate_signature == false`); an in-function magic-gate count does not separate them from category 1 (the interlocking checks live in *callee* functions). So they fold into category 1, confirmed — like the floating-offset sub-case — by the Layer-1 cmplog > naive advantage. (*Folded 2026-06-25.*)

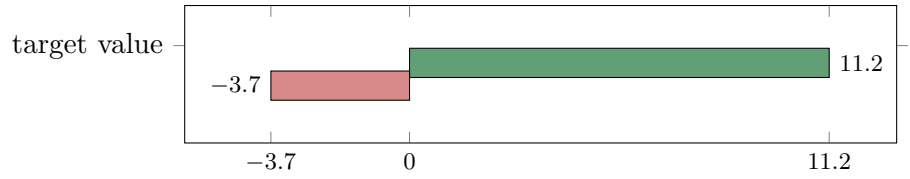
2. Exact number / enum gate `i2s_numeric_tag_substitution` ($n = 39$)

Same move, but the thing matched is a small *number* — a version code, an enum, a `switch`-case selector — not a string. I2S substitutes the precise value. We split it from category 1 only because “it’s a number” vs “it’s a string” matters when you later try to *recognize* the gate; the mechanism (paste the exact value) is identical.

```
// version/enum dispatch -- gate needs one exact constant
switch (header.version) {
  case 2:  parse_v2(...); break;    // <-- blocker: only the value 2
         reaches it
  default: reject();
}
```

How we know. The cmplog corpus over-represents the *target* constant and *under*-represents the competing “decoy” values, exactly as deterministic substitution of one logged operand would predict.

Tool: <code>operand_enrichment</code>
Metric: <code>signed_target_enrich</code> (target value, $\log_2$ ratio) and <code>decoy_enrich</code> (the competing value)
Verify rule (the arbiter fires this, no LLM): <code>signed_target_enrich >= 1.0 AND decoy_enrich <= 0.0</code>
Real example: <code>bloaty/34</code> , switch-case constant — <code>signed_target_enrich= 11.2</code> (target piled up) while <code>decoy_enrich= -3.7</code> (wrong values drained).



log₂ enrichment in the emplog corpus (right = piled up, left = drained)

I2S concentrates the corpus on the one right number and steers it away from the wrong ones.

At a glance

Category	<i>n</i>	Tool / metric	One-line reason
string / literal gate	199	operand.enrichment / signed_target_enrich	paste the exact (incl. 3 floating gate_signature=false , 6 distance-route, 8 multi container)
numeric / enum gate	39	operand.enrichment / signed_target_enrich+decoy_enrich	paste the exact number
I2S-pro total	238		

Counts and example metrics regenerated from `bench/dataset.jsonl`; verify rules are the deterministic arbiter rules (no LLM in the decision).